Lab 6: Code and Branch Coverage Analysis

M Naga Sai Abhiram · PES1UG24AM151 · Software Engineering Lab

`loan_eligibility.py`

1. Setup

The module `loan_eligibility.py` has three functions: `credit_band(score)`, `is_eligible(age, income, employment, score)` and `interest_rate(score, employment, has_collateral)`. It has 33 statements and 24 branches (12 decisions, two outcomes each).

Environment: Python 3.11.9, pytest 9.2.0, pytest-cov 7.1.0, coverage.py. Two test files were written, one aimed at code (line) coverage and one at branch coverage.

2. Code Coverage Analysis

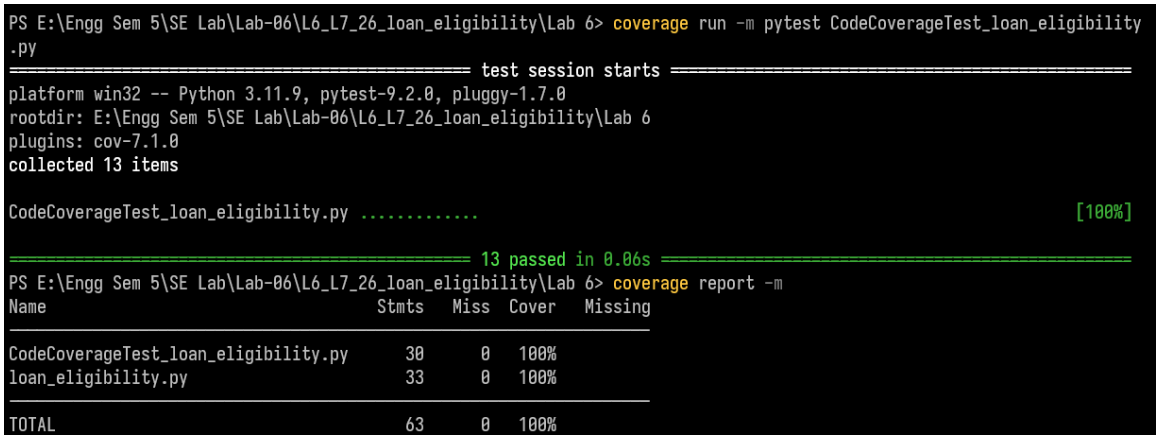
Test file: `CodeCoverageTest_loan_eligibility.py` (13 tests, all passed). Commands:

```
coverage run -m pytest CodeCoverageTest_loan_eligibility.py
coverage report -m
```

The tests were chosen so that every line runs at least once, using one test per path:

Function	Tests	Lines reached
`credit_band`	invalid score, excellent, good, fair, poor	raise, all four return statements
`is_eligible`	unknown employment, bad age, low income, poor credit, success	all four early exits and the final return
`interest_rate`	excellent, good, fair (else)	all three rate assignments, surcharge and discount lines

Result: 33 statements, 0 missed, 100% line coverage.



3. Code Coverage Tests Measured With Branch Coverage

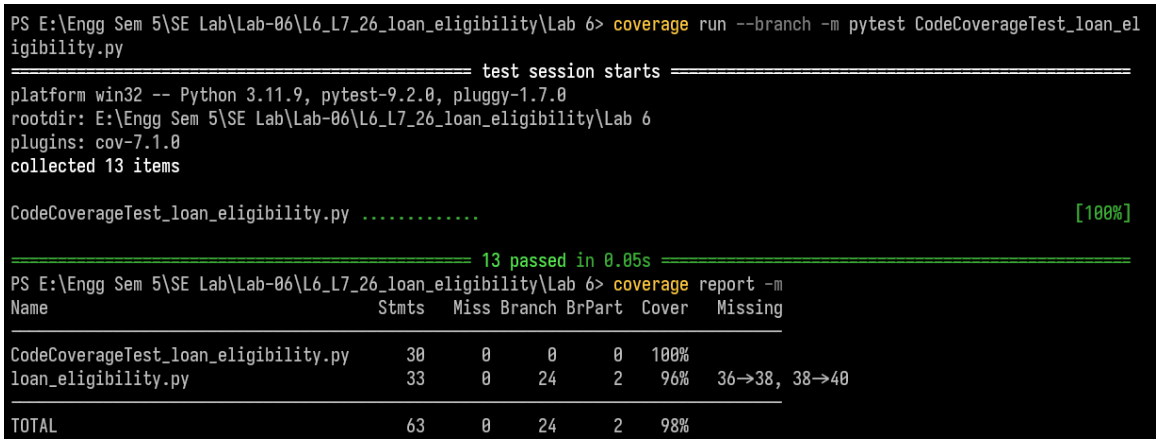
The same 13 tests were re-run with the `--branch` flag:

```
coverage run --branch -m pytest CodeCoverageTest_loan_eligibility.py
coverage report -m
```

Result: 24 branches, 2 partially covered, 96% coverage. Missing: `36->38` and `38->40`.

- `36->38`: the `if employment == "self_employed"` decision never took its false outcome, because every `interest_rate` test used `self_employed`.
- `38->40`: the `if has_collateral` decision never took its false outcome, because every `interest_rate` test passed `True`.

Every line executed, yet two decision outcomes were never exercised. This is the gap between line and branch coverage: 100% line coverage does not mean every decision was tested both ways.



```
PS E:\Engg Sem 5\SE Lab\Lab-06\L6_L7_26_loan_eligibility\Lab 6> coverage run --branch -m pytest CodeCoverageTest_loan_el
igibility.py
============================= test session starts =============================
platform win32 -- Python 3.11.9, pytest-9.2.0, pluggy-1.7.0
rootdir: E:\Engg Sem 5\SE Lab\Lab-06\L6_L7_26_loan_eligibility\Lab 6
plugins: cov-7.1.0
collected 13 items

CodeCoverageTest_loan_eligibility.py .............                       [100%]

============================== 13 passed in 0.05s ==============================
PS E:\Engg Sem 5\SE Lab\Lab-06\L6_L7_26_loan_eligibility\Lab 6> coverage report -m
Name                                 Stmts   Miss Branch BrPart  Cover   Missing
--------------------------------------------------------------------------------
CodeCoverageTest_loan_eligibility.py    30      0      0      0   100%
loan_eligibility.py                     33      0     24      2    96%   36->38, 38->40
--------------------------------------------------------------------------------
TOTAL                                   63      0     24      2    98%
```

4. Branch Coverage Analysis

Test file: `BranchCoverageTest_loan_eligibility.py` (37 tests, all passed). Commands:

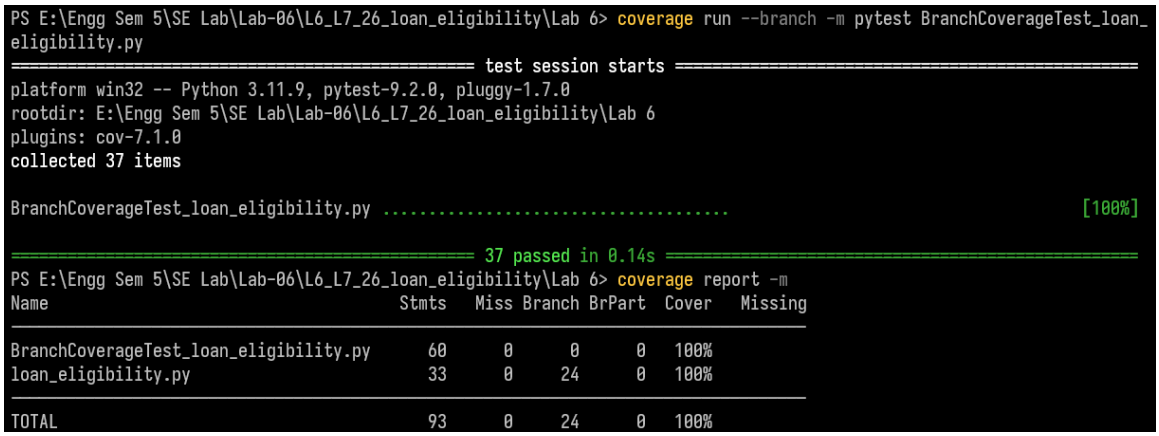
```
coverage run --branch -m pytest BranchCoverageTest_loan_eligibility.py
coverage report -m
```

Every decision was tested for both its true and false outcome, with boundary values around each threshold:

Decision	True case	False case
`score < 300 or score > 900`	299, 901 raise `ValueError`	valid scores 300 to 900
`score >= 750` / `>= 650` / `>= 550`	750, 650, 550	749, 649, 549
`employment not in MIN_INCOME`	unknown type raises	salaried, self_employed
`age < 21 or age > 60`	20, 61 return False	21, 60 pass

Decision	True case	False case
income < MIN_INCOME[employment]	24999 salaried, 39999 self_employed	25000, 40000
credit_band(score) == "poor"	score 500 returns False	score 550 passes
excellent / good / else rate	8, 10, 13 base rates	fair and poor take the else
employment == "self_employed"	+1 surcharge	salaried, no surcharge
has_collateral	-2 discount	no discount

Result: 33 statements, 0 missed, 24 branches, 0 partial, 100% line and 100% branch coverage.



5. Comparison

Test file	Tests	Line coverage	Branch coverage
CodeCoverageTest_loan_eligibility.py	13	100%	96% (2 partial)
BranchCoverageTest_loan_eligibility.py	37	100%	100%

6. Conclusion

Both test files exceed the 90% requirement. Branch coverage is the stricter metric: it exposed the untested self_employed and has_collateral false outcomes that line coverage hid. The branch test file closes that gap and reaches 100% on both metrics. The two test files were committed separately, one for code coverage and one for branch coverage.